Conecta Nexus: Guía Metodológica y Operativa de GitHub Projects (Transición desde Jira)

1. Introducción a GitHub Projects

GitHub Projects (versión 2) es la herramienta nativa de gestión ágil que unifica el seguimiento de tareas con el código fuente, los repositorios y la infraestructura de software. En proyectos basados en arquitecturas de microservicios como Conecta Nexus, donde conviven múltiples repositorios independientes (auth, profile, challenge, application, moderation, notification, frontend, gateway e infra), GitHub Projects permite consolidar un único tablero de control a nivel de organización.

A diferencia de herramientas externas, GitHub Projects elimina la fricción entre la planificación y el desarrollo: los issues, pull requests, revisiones de código y despliegues interactúan de forma nativa e inmediata sin necesidad de sincronizaciones manuales o plugins de terceros.

2. Similitudes y Equivalencias con Jira

Para el equipo de desarrollo, acostumbrado a los flujos de Jira Software, la transición se fundamenta en las siguientes equivalencias directas:

Concepto en Jira	Equivalente en GitHub Projects	Descripción y Configuración en Conecta Nexus
Jira Issue (Story / Task / Bug)	GitHub Issue	Unidad básica de trabajo en cada repositorio con etiquetas de tipología (labels: user-story, task, bug).
Jira Epic	GitHub Issue con etiqueta "epic" / Sub-issues	Contenedor de alto nivel que agrupa requerimientos (ej. EP-01 Identidad, EP-02 Problemáticas).
Jira Board (Kanban / Scrum)	Project Board (Vistas Board / Table / Roadmap)	Tablero visual interactivo filtrado por estado, asignado o

Concepto en Jira	Equivalente en GitHub Projects	Descripción y Configuración en Conecta Nexus
		iteración.
Workflow States	Campo Status (Single Select)	Columnas simplificadas del flujo: To Do, In Progress, In Review y Done.
Story Points	Campo numérico "Story Points"	Estimación basada en Fibonacci (1, 2, 3, 5, 8, 13) para medir esfuerzo relativo.
Jira Sprint	Campo de iteración "Sprint" (Iteration)	Bloques de 4 semanas configurados con fechas de inicio y fin para medir capacidad (aproximadamente 33-34 PH por sprint).
Component	Campo "Repository" / Labels por servicio	Identifica a qué microservicio o componente pertenece el artefacto.
Releases / Fix Version	Milestones / Git Releases	Agrupador temporal o de versión para entregas de corte evaluativo.

Ventajas clave de la migración: Cero cambio de contexto al trabajar en el IDE y terminal, trazabilidad nativa entre código e ítems, y eliminación del costo de licencias externas.

3. Operación en la Organización y Flujo Multi-Repositorio

Para lograr un flujo natural y cohesivo en toda la organización:

1. **Nivel Organizacional:** El proyecto se crea en la pestaña Proyectos de la organización conecta-nexus con el nombre "Conecta Nexus - Core Management", asegurando visibilidad transversal de todos los microservicios.
2. **Vinculación de Repositorios (Linked Repositories):** En la configuración del proyecto (Settings > Linked repositories), se enlazan todos los repositorios del ecosistema: auth, profile, challenge, application, moderation, notification, frontend, gateway e infra.
3. **Ciclo de Vida Integrado:**

    - Los Issues se crean asignados a su repositorio correspondiente pero asociados al Proyecto Organizacional.
    - Todo desarrollador consulta un único tablero central para conocer sus prioridades, independientemente del repositorio en el que deba programar.

4. Creación y Estructura de Ítems (Historias, Tareas y Puntos)

En GitHub Projects conviven dos tipos de ítems:

- **Draft Items:** Notas rápidas en texto plano (útiles para lluvia de ideas durante la planeación del sprint).
- **Issues Formales:** Ítems vinculados a un repositorio concreto (obligatorios para cualquier tarea ejecutable).

Paso a paso para crear un ítem formal:

1. Desde el tablero de GitHub Projects, hacer clic en el botón para agregar ítem al final de la columna "To Do" o presionar la tecla de acceso rápido.
2. Escribir el título preliminar y luego seleccionar la opción de convertir a issue ("Convert to issue"). Seleccionar el repositorio destino (por ejemplo, el repositorio auth).
3. Abrir el Issue y completar la ficha estandarizada con la siguiente estructura:
    - **Título:** `[Módulo/ID]: Acción concreta` (ejemplo: `[AUTH] HU-01: Registro y Asignación de Roles`).
    - **Descripción:**
        - **Historia:** Como `[rol]`, quiero `[funcionalidad]` para `[beneficio]`.
        - **Criterios de Aceptación:** Escenarios Dado / Cuando / Entonces en formato Gherkin.
        - **Checklist Técnico:** Lista de tareas específicas con casillas verificables.
4. Asignar los campos en el panel lateral derecho del ítem:
    - **Status:** To Do
    - **Assignees:** Pareja de desarrolladores responsable (Code Owners).
    - **Sprint:** Iteración activa (Sprint 1, Sprint 2 o Sprint 3).

- **Story Points:** Puntuación acordada en la sesión de planeación (ejemplo: 3, 5 u 8 PH).

5. Reporte y Trazabilidad desde los Repositorios

Los desarrolladores no necesitan mover manualmente las tarjetas en la mayoría de los casos; el avance se reporta orgánicamente a través de Git y GitHub:

1. **Creación de la Rama:**
   - El desarrollador se posiciona en la rama `develop` y crea su rama con referencia al issue o historia:
     `git checkout -b feature/HU-01-auth-jwt` (o `feature/12-auth-jwt`).
2. **Commits Trazables:**
   - Mensajes bajo estándar Conventional Commits referenciando el identificador del issue:
     `git commit -m "feat(auth): implementar hash bcrypt y emision de token #12"`
3. **Transición a "In Progress":**
   - Al comenzar a codificar, el desarrollador arrastra la tarjeta en el tablero a la columna "In Progress" o se asigna a sí mismo.
4. **Apertura del Pull Request y Transición a "In Review":**
   - Se crea el Pull Request con rama base en `develop` y se incluye en la descripción la palabra clave de cierre:
     `Closes #12`
   - Se vincula el Pull Request al proyecto Conecta Nexus y se mueve el estado a "In Review".
   - El revisor asignado (compañero par según la matriz de Code Owners) realiza la revisión de código, resuelve conversaciones y aprueba el cambio.

6. Definición de Terminado (DoD) y Cierre de Sprints

Cierre Automático de Tareas:

Cuando el Pull Request es aprobado y se ejecuta la fusión (Merge) hacia la rama `develop`, GitHub ejecuta dos acciones automáticas:

1. El Issue vinculado (#12) se cierra inmediatamente.
2. La tarjeta correspondiente en el tablero de GitHub Projects se desplaza de forma automática a la columna "Done".

Criterios de Aceptación Técnicos (Definition of Done - DoD):

Un ítem solo se considera finalizado si cumple con:

- Código implementado conforme a los criterios de aceptación de la historia de usuario.
- Pruebas unitarias automatizadas superadas con cobertura mínima del 80% (Pytest para microservicios Python / Vitest para frontend).
- Pipeline de Integración Continua (CI) en verde sin advertencias bloqueantes de linter.
- Aprobación formal de al menos un revisor del equipo (Pull Request aprobado).
- Contratos OpenAPI o esquemas de base de datos actualizados si hubo modificaciones en endpoints o modelos.

Cierre del Sprint:

1. Al cumplirse la fecha límite del Sprint (iteración de 4 semanas), el equipo realiza la sesión de revisión de corte.
2. Los ítems en columna "Done" consolidan los Puntos de Historia completados frente a la meta fijada (33-34 PH por sprint).
3. Si algún ítem queda incompleto en "In Progress" o "In Review", se evalúa en la retrospectiva técnica y se reasigna a la siguiente iteración o se regresa al Backlog general.

7. Métricas, Reportes y Visualización de Insights

GitHub Projects incluye un motor de analítica integrado que sustituye los reportes tradicionales de Jira:

1. **Acceso a Insights:**
    - En el menú superior del proyecto, ingresar a la pestaña "Insights".
2. **Gráficos Clave para Seguimiento y Cortes Evaluativos:**
    - **Burnup / Burndown Chart:** Muestra la acumulación de Story Points completados en el tiempo frente a la capacidad comprometida del sprint.

    - **Gráfico de Velocidad:** Gráfico de barras agrupado por iteración (Sprint 1, Sprint 2, Sprint 3) sumando el campo numérico Story Points para medir el ritmo de entrega del equipo.
    - **Distribución de Trabajo por Asignado:** Gráfico de barras agrupado por Assignees sumando Story Points, útil para verificar el balance de carga entre los integrantes del equipo.
    - **Esfuerzo por Microservicio:** Gráfico de barras agrupado por Repository para transparentar la complejidad dedicada a cada servicio (ejemplo: `challenge` con 33 PH frente a `notification` con 8 PH).
3. **Exportación y Sustentación:**
    - Los tableros permiten filtrar por sprint activo y exportar los datos o presentar las métricas interactivas directamente al docente evaluador durante las revisiones de avance.